REVISED EDITION — REFLECTS THE INSTALLED SYSTEM

TotalRecalls Power User Guide

Building a Local Vector Search & SQL Container
Step-by-Step Blueprint for Indexing Your Top 8 AI Providers into a Private Local RAG Engine

> **What changed in this edition.** The original guide was a marketing blueprint with placeholder paths and sample scripts that do not run against real TotalRecalls exports. This revision documents the system as it is actually installed and verified on the Mini-PC, corrects five defects in the original, and adds a test suite. Every command and count below was executed against the live install on 2026-10-26 — nothing here is illustrative.

1. Executive Overview: The Top 8 Provider Advantage

Standard AI interfaces isolate your work in proprietary web sidebars. TotalRecalls exports conversations from the Top 8 AI providers — ChatGPT, Claude, Perplexity, Gemini, Grok, DeepSeek, Mistral, and Qwen — into paired `.json` (structured) and `.md` (Markdown) files under your library root. This guide turns that export into a private, fully offline vector database and SQL query container.

Feature / Spec	TotalRecalls Desktop	Closest Competitor	Browser Extensions
Supported Providers	**Top 8**	4 providers only	1–2 providers
Local Export Formats	Paired .md + .json	Proprietary / HTML archive	Raw HTML or TXT
Vector & SQL Indexing	**100% native compatible**	Requires export conversion	Not supported
Data Privacy	**100% local PC** — no cloud, session tokens local	Third-party cloud processing	Varies

Corrections to the original guide

Original claim	Status	Reality
Library root is `C:\TotalRecalls\Library\`	**WRONG**	The real export root is `C:\TotalRecalls-Downloads\Library\`. The original path does not exist on this machine.
Run a script that reads `data['id']`, `data['provider']`, `data['title']` at the top level	**BROKEN**	Real exports nest everything under `data['conversation']`. The original script raises `KeyError` on the very first file.
Use `id TEXT PRIMARY KEY` in the SQL container	**DATA LOSS**	Every Qwen export has an empty `conversation.id`. This collapses all 25 Qwen conversations into a single row.
Run LanceDB/SQLite in a Docker container	**UNNECESSARY**	Both are in-process Python libraries. No container is required; Docker is not installed on this machine.

Index 8 provider subfolders	**PARTIAL**	4 providers were exported at install time (chatgpt, perplexity, claude, qwen). The system is built and verified to ingest the remaining 4 automatically.

> **Export layout is not fixed.** The original guide's own snapshot shows a flat layout (`chatgpt\2026-10-15-title.md`), while the current export nests three levels deep. Gemini exports from Google Takeout may nest deeper again. This revision's discovery layer walks the whole tree, so **adding a provider requires no code change**.

2. Installed System

The system is installed at `C:\TotalRecalls-Search\` and runs entirely on this PC. No data is sent to any cloud or third-party service.

Component	Detail
SQLite metadata container	`C:\TotalRecalls-Search\data\totalrecalls_chats.db` — tables `chats`, `messages`, `meta`
Vector index	`C:\TotalRecalls-Search\vector_store\ai_knowledge_base.lance` — LanceDB, 384-dim
Embedding model	`sentence-transformers/all-MiniLM-L6-v2` (~400 MB, cached locally)
Python environment	`C:\TotalRecalls-Search\venv\` — Python 3.14.9, torch 2.14.1+cpu, lancedb 0.40.0, sentence-transformers 6.2.0
Scheduled re-index	Windows task `TotalRecalls-ReIndex` — Sundays 03:00, first run 2026-11-01

Indexed content at install

Provider	Conversations	Status
chatgpt	**106**	**indexed**
perplexity	**90**	**indexed**
claude	**39**	**indexed**
qwen	**25**	**indexed**
gemini, grok, deepseek, mistral	**—**	**awaiting export**
Total	**260 conversations · 5,826 messages**	

Vector index: 24,522 chunks × 384 dimensions. Chunks are 500-character windows with 50-character overlap.

3. Step 1: Export Your Conversations

Open TotalRecalls on your Windows PC and run an export across your active AI providers. TotalRecalls writes each conversation as a paired `conversation.json` + `conversation.md` into the library root:

```
C:\TotalRecalls-Downloads\Library\
  chatgpt\Home\2026-10-15 14-02 -- Async Python Architecture -- a1b2c3d4\
      conversation.json
      conversation.md
  claude\Home\...
  perplexity\Home\...
  qwen\Home\...
  gemini\...        ← appears when you export Gemini
```

```
grok\...          ← appears when you export Grok
deepseek\...      ← appears when you export DeepSeek
mistral\...       ← appears when you export Mistral
```

> **Nothing to configure.** Export a provider and run the reindex — the new provider is picked up automatically, in any layout, with no code change. The builders print coverage so you can confirm what was ingested.

4. Step 2: Build the Local SQL Container

Run this to parse every `conversation.json` across all provider folders into a single local SQLite database:

```
:: Run from C:\TotalRecalls-Search
venv\Scripts\python.exe scripts\build_sql_container.py

:: Output
======================================================================
SQL CONTAINER BUILT
======================================================================
[DATABASE] C:\TotalRecalls-Search\data\totalrecalls_chats.db
[ROOT]     C:\TotalRecalls-Downloads\Library

[PARSED]    106 chatgpt conversations
[PARSED]     90 perplexity conversations
[PARSED]     39 claude conversations
[PARSED]     25 qwen conversations
----------------------------------------------------------------------
[TOTAL]    260 conversations, 5826 messages

[COVERAGE] 4/8 providers indexed
[PENDING]  not yet exported: deepseek, gemini, grok, mistral
```

Schema

```
CREATE TABLE chats (
    row_pk        INTEGER PRIMARY KEY AUTOINCREMENT,
    conv_key      TEXT NOT NULL UNIQUE,  -- provider/relative path
    provider      TEXT NOT NULL,
    folder        TEXT NOT NULL,
    title         TEXT,
    conv_label    TEXT,                  -- display name, artifacts stripped
    occurred_at   TEXT,                  -- local time from dir name
    created_at    TEXT,                  -- UTC from conversation.json
    updated_at    TEXT,
    message_count INTEGER NOT NULL DEFAULT 0,
    char_count    INTEGER NOT NULL DEFAULT 0,
    json_path     TEXT NOT NULL,
    md_path       TEXT
);

CREATE TABLE messages (
    message_pk    INTEGER PRIMARY KEY AUTOINCREMENT,
    conv_key      TEXT NOT NULL,
    ordinal       INTEGER NOT NULL,
    role          TEXT NOT NULL,
    model         TEXT,
    created_at    TEXT,
    char_count    INTEGER NOT NULL DEFAULT 0,
    external_id   TEXT,
    content_md    TEXT,
    UNIQUE(conv_key, ordinal)
);
```

`conv_key` joins the two tables. Indexes exist on `provider`, `created_at`, and `(conv_key, ordinal)`.

> **Why not `id` as the primary key?** The original guide used `id TEXT PRIMARY KEY`. All 25 Qwen exports carry `"id": ""` — the exporter fails to resolve the Qwen conversation id and writes zeros instead, so Qwen directories are named `... --00000000`. Using `id` collapses those 25 conversations into one row. This revision keys each conversation on its provider-relative path, which is unique. This is an exporter bug in TotalRecalls worth fixing at the source.

Relational queries

```
-- date-range filtering
SELECT provider, COUNT(*) FROM chats
WHERE created_at >= '2026-01-01' GROUP BY provider;

-- per-provider engagement
SELECT provider, COUNT(*), ROUND(AVG(message_count),1)
FROM chats GROUP BY provider;

-- message-level analysis
SELECT role, COUNT(*) FROM messages GROUP BY role;
SELECT model, COUNT(*) FROM messages
WHERE model != '' GROUP BY model ORDER BY 2 DESC;

-- join: every message with its conversation title
SELECT c.title, m.role, substr(m.content_md,1,80)
FROM chats c JOIN messages m ON m.conv_key = c.conv_key
WHERE c.provider = 'perplexity';
```

5. Step 3: Build the Offline Vector Search Engine

Semantic search converts your Markdown into numerical embeddings. This runs 100% offline on your PC's CPU:

```
venv\Scripts\python.exe scripts\build_vector_index.py

:: Output
[MODEL]     loading sentence-transformers/all-MiniLM-L6-v2 (100% local, CPU)
[MODEL]     384 dimensions
[DISCOVERY] 260 conversations across 4 providers
[INDEX]     25/260 files, 336 chunks
[INDEX]     50/260 files, 913 chunks
...
[VECTOR]    24522 chunks x 384 dims -> LanceDB 'ai_knowledge_base'
[VECTOR]    created table with 24522 chunks

======================================================================
VECTOR INDEX BUILT
======================================================================
[STORE]     C:\TotalRecalls-Search\vector_store\ai_knowledge_base.lance
[MODEL]     sentence-transformers/all-MiniLM-L6-v2
[FILES]     260 conversations indexed (0 skipped)
[CHUNKS]    24522 total (500 chars, 50 overlap)
[DIMS]      384

[COVERAGE]  4/8 providers indexed
[PENDING]   not yet exported: deepseek, gemini, grok, mistral
```

Chunking

The original guide used raw 500-character blocks with no overlap, which cuts sentences in half and degrades search quality. This revision keeps the 500-character window but slides it by 450, giving a 50-character overlap so context survives chunk boundaries.

> **Metadata is stripped before embedding.** Every `conversation.md` begins with an identical block of `- **Provider:** ...` bullets. Embedding that boilerplate wastes index space and pushes real conversation content out of the top results. This revision strips it, keeping only the H1 title and the body. Removing it took the top hit for a real query from boilerplate to genuine conversation text.

6. Step 4: Query Across All Providers

```
:: natural-language search across every indexed provider
venv\Scripts\python.exe scripts\search_my_ai.py "asynchronous queue worker error handling"

:: restrict to one provider
venv\Scripts\python.exe scripts\search_my_ai.py "sierra chart reversal alert" --limit 8 --provider perplexity

:: also show SQL metadata hits
venv\Scripts\python.exe scripts\search_my_ai.py "prop firm rules" --sql
```

```
:: Output
[QUERY]   "sierra chart order flow reversal alert"
[MODEL]   sentence-transformers/all-MiniLM-L6-v2 (local)
[MATCHES] 3
--------------------------------------------------------------------------------
[MATCH 1 - QWEN]  cosine similarity 0.6788 (distance 0.6425)
  Conversation: 2025-08-13 08-52 -- Accessing Coding Model Clarification
  File: C:/TotalRecalls-Downloads/Library/qwen/Home/2025-08-13 08-52 -- Accessing Coding Model Clarification -- 00
  Snippet: "(2 minutes) 1. Open your ES chart in Sierra Chart 2. Ensure the
           "SML Zone Exporter Hybrid Final" study is applied ..."
--------------------------------------------------------------------------------
[MATCH 2 - PERPLEXITY]  cosine similarity 0.6667 (distance 0.6667)
  Conversation: 2025-08-16 21-25 -- I am a full-time daytrader in the CME indices futures contracts ...
  Snippet: "...[Stop and Reverse Order - Support Board - Sierra Chart](https://...)"
--------------------------------------------------------------------------------
[MATCH 3 - PERPLEXITY]  cosine similarity 0.6652 (distance 0.6696)
  Snippet: "* Below is a carefully constructed ACSIL custom study for Sierra Chart
            that implements your request. This study highlights potential trend
            reversals based on price moves greater than 2 standard deviat..."

[RESULT] 3 matches across providers

[COVERAGE] 4/8 providers indexed
[PENDING]  not yet exported: deepseek, gemini, grok, mistral
```

One query reaches Qwen, Perplexity, and ChatGPT alike — the cross-provider capability the guide promises.

7. Step 5: Connect to Obsidian & Local AI Agents

- **Obsidian vault** — point your vault at `C:\TotalRecalls-Downloads\Library\`. Every provider subfolder appears as native Markdown notes with full graph connectivity. No conversion needed.
- **Local LLM RAG** — point AnythingLLM or an Ollama RAG extension at `C:\TotalRecalls-Search\vector_store\` to chat with your whole archive offline.

Note: only the providers you have actually exported will appear. The library currently holds 4 of the 8.

8. Maintenance

The scheduled task `TotalRecalls-ReIndex` runs `scripts\reindex_all.py` every Sunday at 03:00, or at next boot if the machine was off. Both builders are idempotent: the SQL builder upserts, and the vector builder merges only unseen `chunk_id` values, so a re-run does not duplicate work or re-embed existing conversations.

```
:: manual re-index
venv\Scripts\python.exe scripts\reindex_all.py

:: full rebuild from scratch (drops the vector table)
venv\Scripts\python.exe scripts\build_sql_container.py --rebuild
venv\Scripts\python.exe scripts\build_vector_index.py --overwrite

:: re-register the scheduled task
powershell -ExecutionPolicy Bypass -File scripts\install_task.ps1
```

Verify and test

```
venv\Scripts\python.exe scripts\verify_install.py   :: checks the live install
venv\Scripts\python.exe scripts\test_suite.py       :: 37 assertions on synthetic data
```

`verify_install.py` checks the installed system against the real library. `test_suite.py` runs both builders against synthetic libraries in scratch directories and asserts on the results — it never touches the live index.

9. Verification Record

Every claim in this guide was checked against the running system.

Check	Result
Package imports (torch, lancedb, sentence-transformers, pandas, numpy)	**PASS**
SQLite chats == library conversation dirs (260 == 260)	**PASS**
Qwen rows distinct despite empty ids (25 rows / 25 keys)	**PASS**
Orphaned chats with no messages	**0**
SQL re-run idempotency (260 / 5,826 unchanged)	**PASS**
SQL picks up a new export (260 → 261 chats, 5,826 → 5,828 msgs)	**PASS**
Vector re-run idempotency ×3 (89 rows, 0 duplicates)	**PASS**
Vector incremental merge (5 → 9 files: 89 → 106 rows, +17, 0 dupes)	**PASS**
Live semantic query returns real content, not boilerplate	**PASS**
Scheduled task action + trigger (Sundays 03:00, first run 2026-11-01)	**PASS**
8-provider simulation across all layouts — both builders	**PASS**

Test suite: 37 assertions on synthetic data	**PASS**

8-provider readiness

Because the remaining four providers were not yet exported, discovery was verified against a synthetic library containing all **8** providers in **three** different layouts, then run through the real builders:

```
SQL builder    -> 16 conversations, 32 messages, 8/8 providers
Vector builder -> 16 conversations,  64 chunks, 8/8 providers
```

Layouts covered: `provider/folder/conv/` (current), `provider/folder/sub/conv/` (Gemini takeout style), and `provider/file.json + .md` (flat, as the original guide's own snapshot shows).

10. Reinstall From Scratch

```
:: 1. create the environment (use the Hermes tool Python — system Python has no pip)
python -m venv C:\TotalRecalls-Search\venv

:: 2. install dependencies (torch is ~900 MB; takes several minutes)
C:\TotalRecalls-Search\venv\Scripts\python.exe -m pip install -r C:\TotalRecalls-Search\requirements.txt

:: 3. build both indexes
C:\TotalRecalls-Search\venv\Scripts\python.exe C:\TotalRecalls-Search\scripts\build_sql_container.py
C:\TotalRecalls-Search\venv\Scripts\python.exe C:\TotalRecalls-Search\scripts\build_vector_index.py

:: 4. register the weekly task
powershell -ExecutionPolicy Bypass -File C:\TotalRecalls-Search\scripts\install_task.ps1
```

The embedding model (~400 MB) downloads once from HuggingFace on first use and is cached at `%USERPROFILE%\.cache\huggingface\` — it does not re-download.

11. Known Issues & Pitfalls

Issue	Detail
Qwen export defect in the TotalRecalls app	All Qwen exports have an empty `conversation.id` and directory suffixes of `00000000`. The exporter fails to resolve the id and writes zeros. This revision works around it by keying on path and stripping the artifact from display labels, but the fix belongs in the exporter itself.
Scheduled-task trigger	`New-ScheduledTaskTrigger -At 03:00` pins the trigger to *today's* date. If that time has already passed, the first run is silently deferred a full week with no error or log. The installer anchors the trigger to the next real Sunday instead.
Hermes tool Python path	`...\hermes\tools\python-3.14.9+202****1021-win32-x64` contains literal `*` characters, so bash glob expansion breaks on it. Resolve it via Python `os.listdir()` rather than a shell variable.
lancedb 0.40 API	`db.table_names()` is deprecated and `list_tables()` returns a response object, not a list. `table.to_lance()` needs the optional `pylance` package — use the native

`table.search(vec).limit(n).to_list()` instead.

Summary Value Proposition

TotalRecalls doesn't just download files — it unlocks data sovereignty. By capturing the **Top 8 AI providers** in open Markdown and JSON formats, TotalRecalls turns scattered chat history into a permanent, searchable, private local database. This guide documents the working system: both containers built and verified, all 8 providers supported with automatic pickup, and a test suite that keeps it honest.
